Incremental Delta Scanning with Selective Re-Embedding for Encrypted Data Indexes

Tom Tang
Edge Vector Foundation

2026

Abstract

A system and method for maintaining encrypted semantic search indexes over personal data through incremental content-hash delta scanning and selective re-embedding. The system walks a directory tree and computes BLAKE3 content hashes for each file, comparing the computed hashes against a stored hash manifest to identify files that have been added, modified, or deleted since the last scan. Only files whose hashes have changed are submitted to a format-aware processing pipeline that extracts text, generates vector embeddings, and indexes the results. The entire index—including vector embeddings, extracted metadata, and cached content representations—is encrypted at rest using an envelope encryption architecture: a user-supplied passphrase is processed through Argon2id key derivation to produce a master key, from which separate Content Key, Metadata Key, and Vector Key are derived via HKDF, each encrypting its respective data partition with AES-256-GCM authenticated encryption. Embeddings are cached in encrypted form alongside their content hashes, enabling hash-based cache invalidation without decrypting the embedding store. The processing pipeline is format-aware and tiered: local parsing for structured formats, vision-model extraction for images, LLM-assisted extraction for PDFs and unknown formats, and chunked parallel processing with merge for large files exceeding a configurable threshold. All processing occurs locally with zero outbound network connections during normal operation. The system optionally integrates with filesystem event APIs (FSEvents on macOS, inotify on Linux) to receive real-time change notifications, eliminating the need for periodic polling. A dual-read migration mode enables transparent transition from plaintext to encrypted storage without downtime or data loss.

1 Introduction

This work relates generally to personal data indexing and semantic search systems, and more particularly to methods and systems for incrementally maintaining encrypted vector indexes over heterogeneous file collections using content-hash-based change detection, selective re-embedding, and hierarchical envelope encryption, wherein all processing occurs locally on the user's device without transmitting personal data to external services.

2 Background

2.1 The Personal Data Indexing Problem

Users accumulate large collections of heterogeneous files—documents, images, code, configuration files, spreadsheets, and notes—across local storage. Semantic search over these collections requires generating vector embeddings that capture the semantic content of each file. However, regenerating

embeddings for an entire file collection on every search or update is computationally expensive and wasteful, as the vast majority of files remain unchanged between indexing passes.

Existing approaches to this problem exhibit several deficiencies:

Full re-indexing. Many personal search tools rebuild the entire index on each pass, recomputing embeddings for all files regardless of whether they have changed. For a collection of N files where $k \ll N$ have changed, this wastes computational resources proportional to $N - k$. As collections grow into the tens or hundreds of thousands of files, full re-indexing becomes prohibitively slow.

Timestamp-based change detection. Some systems use file modification timestamps to detect changes. However, timestamps are unreliable: file copies, backups, version control operations, and filesystem migrations can alter timestamps without changing content, or preserve timestamps despite content changes. Timestamp granularity varies across filesystems (some report only second-level precision), leading to missed changes within the same second.

Unencrypted indexes. Most personal search and indexing tools store vector embeddings, extracted text, and metadata in plaintext. An adversary who gains access to the index—through device theft, backup compromise, or unauthorized filesystem access—can reconstruct the semantic content of the user's files without accessing the files themselves. Vector embeddings, while not human-readable, can be used for nearest-neighbor attacks to infer document topics, and extracted metadata and text are directly readable.

Monolithic encryption. Systems that do encrypt their indexes typically use a single encryption key for the entire store. This prevents granular key rotation (rotating the metadata key without re-encrypting embeddings), complicates access control (a single key grants access to everything), and creates a single point of compromise.

Cloud-dependent processing. Many semantic search systems send user data to cloud APIs for embedding generation. This exposes personal data to third-party services, introduces network latency, incurs per-request costs, and fails when the network is unavailable. For sensitive personal data—medical records, financial documents, private correspondence—cloud transmission may violate privacy requirements or user preferences.

2.2 Limitations of Prior Art

Desktop search engines (Spotlight, Windows Search, Recoll) index file metadata and full text but do not generate vector embeddings for semantic search. Their change detection relies on filesystem event monitoring, which misses changes that occurred while the monitor was not running, and they do not encrypt their indexes.

Vector databases (Pinecone, Weaviate, Milvus, Qdrant) provide efficient similarity search over embeddings but do not handle change detection, file parsing, or embedding generation. They are designed as servers, not local-first personal tools. Most do not encrypt stored vectors at rest with user-controlled keys.

Retrieval-augmented generation (RAG) frameworks (LangChain, LlamaIndex) provide pipelines for parsing, chunking, embedding, and indexing documents. However, they perform full re-indexing by default, do not implement content-hash-based delta scanning, do not encrypt the resulting indexes with user-controlled keys, and rely on cloud embedding APIs.

File synchronization systems (rsync, Syncthing) use content hashing for efficient delta transfer but do not generate embeddings, do not build searchable indexes, and do not encrypt their hash manifests.

Encrypted search systems in the academic literature (searchable encryption, encrypted indexes) focus on enabling search over encrypted data stored on untrusted servers. They do not

address the problem of incrementally maintaining the encrypted index as the underlying plaintext files change, do not integrate format-aware parsing pipelines, and do not optimize for the local-first case where the encryption protects data at rest on the user's own device.

None of these approaches provide a unified system that combines content-hash delta scanning for incremental change detection, selective re-embedding of only changed files, format-aware tiered processing, hierarchical envelope encryption with separate keys for different data partitions, and fully local operation with zero outbound network connections.

3 SUMMARY

The present invention provides a system and method for incrementally maintaining encrypted semantic search indexes over personal data collections. The invention comprises:

1. **Content-hash delta scanning.** The system walks a directory tree and computes a BLAKE3 cryptographic hash for each file. The computed hashes are compared against a stored hash manifest from the previous scan. Files are classified into four categories: unchanged (hash matches), modified (hash differs), added (no prior hash exists), and deleted (prior hash has no corresponding file). Only files classified as modified or added are submitted to the processing pipeline. Deleted files have their index entries and cached embeddings removed. Unchanged files retain their existing embeddings without any recomputation.

2. **Selective re-embedding with persistent cache.** Vector embeddings are stored in an encrypted cache keyed by content hash. When a file's content hash matches the cache, the cached embedding is used directly without invoking the embedding model. When a file's content hash has changed, the old cache entry is invalidated and a new embedding is generated and cached under the new hash. This cache is persistent across system restarts and survives index rebuilds, as any file whose content has been previously embedded—even if deleted and later restored—will hit the cache.

3. **Envelope encryption with hierarchical key derivation.** The user provides a passphrase that is processed through the Argon2id memory-hard key derivation function to produce a 256-bit master key. From this master key, three purpose-specific subkeys are derived using HKDF (HMAC-based Key Derivation Function) with distinct context strings:

   (a) **Content Key**: encrypts extracted text, parsed content, and chunked representations.

   (b) **Metadata Key**: encrypts file paths, modification times, format classifications, and hash-to-file mappings.

   (c) **Vector Key**: encrypts vector embeddings and similarity index structures.

   Each subkey encrypts its data partition using AES-256-GCM with unique nonces per record. Subkeys can be rotated independently: rotating the Metadata Key does not require re-encrypting embeddings or content.

4. **Format-aware tiered processing pipeline.** Files are classified by format and routed to the appropriate processing tier:

   (a) **Tier 1 (Local Parse)**: Structured formats (JSON, CSV, YAML, TOML, XML) are parsed locally using deterministic parsers with no model invocation.

(b) **Tier 2 (Vision Model)**: Image formats (PNG, JPEG, TIFF, WebP) are processed through a local vision model to extract descriptions and text content.

(c) **Tier 3 (Text + LLM)**: PDF documents and unknown formats are processed through text extraction followed by a local LLM for structure recognition and semantic summarization.

(d) **Tier 4 (Chunked Parallel)**: Files exceeding a configurable size threshold (default 128KB) are split into chunks, processed in parallel through the appropriate tier, and merged into a coherent representation.

5. **Semantic-boundary-aware chunking.** Files that require chunking are split along semantic boundaries appropriate to their format: function and class boundaries for source code (detected via tree-sitter parsing), paragraph and section boundaries for prose, record boundaries for structured data, and slide or page boundaries for presentations. Chunks include configurable overlap regions to preserve cross-boundary context.

6. **Filesystem event integration.** The system optionally registers with the operating system's filesystem event API (FSEvents on macOS, inotify on Linux, ReadDirectoryChangesW on Windows) to receive real-time notifications of file creation, modification, and deletion. When filesystem events are available, the system processes changes incrementally without performing a full directory walk, falling back to full delta scanning only when the event queue overflows or after system restart.

7. **Zero outbound network operation.** During normal indexing and search operation, the system makes no outbound network connections. Embedding models, vision models, and LLMs run locally. The hash manifest, embedding cache, and encrypted index are stored on local disk. Network access is neither required nor attempted.

8. **Dual-read migration mode.** When transitioning from plaintext to encrypted storage, the system operates in a dual-read mode: reads attempt decryption first; if decryption fails (indicating a plaintext record), the record is read as plaintext, immediately re-encrypted, and written back. This enables transparent, incremental migration without downtime, without a separate migration tool, and without risk of data loss if the migration is interrupted.

4 DETAILED DESCRIPTION

4.1 1. System Architecture Overview

The system comprises six components (see FIG. 1):

(a) A directory scanner that traverses the monitored directory tree, computes BLAKE3 content hashes for each file, and compares them against the stored hash manifest to produce a change set of added, modified, and deleted files.

(b) A format classifier that examines each changed file's extension, magic bytes, and initial content to determine its format and assign it to the appropriate processing tier.

(c) A tiered processing pipeline that extracts text, metadata, and semantic content from each changed file according to its assigned tier, using local parsers, vision models, or LLMs as appropriate.

(d) An embedding engine that converts extracted text and semantic content into vector embeddings using a locally-running embedding model. The engine checks the encrypted embedding cache before computing new embeddings and writes new embeddings to the cache upon generation.

(e) An encrypted index store that persists the vector index, metadata index, content store, hash manifest, and embedding cache, with each partition encrypted under its respective subkey derived from the master key.

(f) A filesystem event monitor that optionally receives real-time change notifications from the operating system and dispatches them to the directory scanner for incremental processing.

4.2 2. Content-Hash Delta Scanning

The delta scanning process operates as follows (see FIG. 2):

4.2.1 2.1 Hash Computation

For each file f in the monitored directory tree, the system computes:

$$h(f) = \text{BLAKE3}(\text{contents}(f))$$

BLAKE3 is chosen for its combination of cryptographic collision resistance and high throughput. On modern hardware, BLAKE3 achieves throughput exceeding 1 GB/s per core through its tree-hashing mode, which enables parallel processing of large files. The hash computation reads the file contents in streaming fashion, requiring constant memory regardless of file size.

4.2.2 2.2 Manifest Comparison

The system maintains a hash manifest M: a mapping from file paths to their most recently computed content hashes:

$$M : \text{Path} \to \text{Hash}$$

After computing $h(f)$ for all files in the current directory tree, the system classifies each file:

- **Unchanged**: $f \in \text{dom}(M)$ and $M(f) = h(f)$. No action required.
- **Modified**: $f \in \text{dom}(M)$ and $M(f) \neq h(f)$. Submitted to the processing pipeline.
- **Added**: $f \notin \text{dom}(M)$. Submitted to the processing pipeline.
- **Deleted**: $p \in \text{dom}(M)$ and no file exists at path p. Index entries and cached embeddings for p are removed.

After processing, the manifest is updated: $M(f) \leftarrow h(f)$ for all processed files, and entries for deleted files are removed.

4.2.3 2.3 Efficiency Analysis

For a collection of N files where k have changed since the last scan, the system performs N hash computations (each streaming and parallelizable), N hash comparisons (constant-time per comparison), and k processing pipeline invocations. The cost of the hash computation phase is $O(S)$ where S is the total size of all files, dominated by I/O bandwidth rather than CPU. The processing phase cost is proportional to k, not N. When $k \ll N$, which is the common case for incremental updates, this represents a substantial reduction in computational cost compared to full re-indexing.

4.3 3. Envelope Encryption Architecture

4.3.1 3.1 Key Derivation Hierarchy

The encryption key hierarchy is derived from a single user-supplied passphrase P (see FIG. 2):

Step 1: Master Key Derivation. The passphrase is processed through Argon2id, a memory-hard key derivation function that resists both GPU-based and ASIC-based brute-force attacks:

$$K_{\text{master}} = \text{Argon2id}(P, \text{salt}, t, m, p)$$

where salt is a 128-bit random value stored alongside the encrypted index, t is the time cost (number of iterations), m is the memory cost (in kilobytes), and p is the degree of parallelism. The default parameters are $t = 3$, $m = 65536$ (64 MB), $p = 4$, producing a 256-bit master key.

Step 2: Subkey Derivation. Three purpose-specific subkeys are derived from K_{master} using HKDF-SHA256 with distinct context strings:

$$K_{\text{content}} = \text{HKDF}(K_{\text{master}}, \texttt{"content-key-v1"}) \tag{1}$$

$$K_{\text{metadata}} = \text{HKDF}(K_{\text{master}}, \texttt{"metadata-key-v1"}) \tag{2}$$

$$K_{\text{vector}} = \text{HKDF}(K_{\text{master}}, \texttt{"vector-key-v1"}) \tag{3}$$

Each subkey is 256 bits. The context strings include a version suffix to support future key rotation: deriving with `"content-key-v2"` produces a completely independent key, enabling re-encryption of a single partition without affecting the others.

4.3.2 3.2 Per-Record Encryption

Each record R in a partition is encrypted as:

$$(N, C, T) = \text{AES-256-GCM.Encrypt}(K_{\text{partition}}, N, R, A)$$

where N is a 96-bit nonce generated from a cryptographically secure random number generator (CSPRNG) conforming to NIST SP 800-90A (unique per record), C is the ciphertext, T is the 128-bit authentication tag, and A is associated authenticated data (authenticated but not encrypted). Nonces are generated using a CSPRNG conforming to NIST SP 800-90A. With 96-bit random nonces and AES-256-GCM, the birthday bound gives collision probability approximately $N^2/2^{97}$ after N encryptions under the same key, which remains negligible for expected operational volumes. The nonce, ciphertext, and tag are stored together.

The associated authenticated data A is partition-specific, binding each encrypted record to its context and preventing record substitution attacks where an attacker could swap encrypted records between files or partitions:

- **Content partition**: $A = (\text{content_hash}, \text{partition_id}, \text{file_path})$, binding the encrypted content to the specific file from which it was extracted.
- **Metadata partition**: $A = (\text{file_path}, \text{partition_id}, \text{content_hash_of_source_file})$, binding metadata to both the file path and its content, preventing an attacker from associating a file's metadata with a different file's content.
- **Vector partition**: $A = (\text{content_hash}, \text{partition_id}, \text{embedding_model_version})$, binding each embedding to both the content it represents and the model version that generated it, ensuring embeddings cannot be substituted between files or across model upgrades.

4.3.3 3.3 Key Rotation

To rotate a subkey (for example, K_{metadata}):

1. Derive the new subkey: $K'_{\text{metadata}} = \text{HKDF}(K_{\text{master}}, \texttt{"metadata-key-v2"})$.
2. Decrypt each metadata record with K_{metadata}.
3. Re-encrypt each metadata record with K'_{metadata}.
4. Update the partition's version marker from `v1` to `v2`.

The content and vector partitions are unaffected. This enables targeted key rotation in response to suspected compromise of a single partition.

4.4 4. Persistent Embedding Cache

The embedding cache maps content hashes to encrypted embeddings:

$$\mathcal{E} : \text{Hash} \to \text{Enc}(K_{\text{vector}}, \text{embedding})$$

When a file with content hash h is submitted for embedding:

1. If $h \in \text{dom}(\mathcal{E})$, the cached embedding is decrypted and used. No model invocation occurs.
2. If $h \notin \text{dom}(\mathcal{E})$, the file is processed through the pipeline, the embedding is generated, encrypted under K_{vector}, and stored in $\mathcal{E}$ keyed by h.

The cache is keyed by content hash, not by file path. This means:

- A file that is renamed without modification hits the cache (same content, same hash).
- A file that is deleted and later restored with identical content hits the cache.
- Two files with identical content share a single cache entry, deduplicating embedding storage.
- Cache invalidation is automatic: when a file's content changes, its hash changes, and the old cache entry is no longer referenced.

Stale cache entries (hashes that no longer correspond to any file in the manifest) are garbage-collected during a periodic compaction pass.

4.5 5. Format-Aware Tiered Processing Pipeline

4.5.1 5.1 Format Classification

Each file is classified by examining, in order: (1) file extension, (2) magic bytes (the first 512 bytes of the file), and (3) initial content heuristics (e.g., detecting JSON by attempting a parse of the first 4KB). The classifier assigns each file to exactly one processing tier.

4.5.2 5.2 Tier 1: Local Parse

Structured data formats—JSON, CSV, TSV, YAML, TOML, XML—are parsed using deterministic, locally-executing parsers. The parser extracts the data structure, flattens nested objects into a key-path representation, and produces a text representation suitable for embedding. No model invocation is required. This tier has the lowest latency and highest throughput.

4.5.3 5.3 Tier 2: Vision Model

Image files are processed through a locally-running vision model (e.g., a quantized multimodal model) that produces a textual description of the image content, extracts any visible text via OCR, and identifies key objects and scenes. The textual output is then embedded. This tier requires GPU or accelerated inference but makes no network calls.

4.5.4 5.4 Tier 3: Text + LLM

PDF documents, office documents, and files of unknown format undergo text extraction (via a local PDF parser or format-specific extractor), followed by processing through a locally-running LLM that identifies document structure, extracts key topics, and produces a semantic summary. The combination of extracted text and LLM summary is embedded. This tier is more computationally expensive than Tiers 1 and 2 but handles the widest range of formats.

4.5.5 5.5 Tier 4: Chunked Parallel Processing

Files exceeding the configurable size threshold (default 128KB of extracted text) are split into chunks along semantic boundaries (see Section 6) and processed in parallel. Each chunk is independently processed through the tier appropriate to its format. After all chunks are processed, the system merges chunk-level embeddings into a document-level embedding using mean pooling: $e_{\text{doc}} = \frac{1}{k}\sum_{i=1}^{k} e_{\text{chunk}_i}$, where k is the number of chunks. This deterministic merge strategy ensures that the same document always produces the same document-level embedding regardless of processing order. Both chunk-level and document-level embeddings are stored for multi-granularity search.

4.6 6. Semantic-Boundary-Aware Chunking

When files require chunking, the system identifies semantic boundaries appropriate to the file's format:

Source code. The system uses tree-sitter grammars to parse the file into an abstract syntax tree and identifies function, method, and class boundaries. Each top-level definition becomes a chunk. Nested definitions are included in their parent's chunk. If a single function exceeds the chunk size, it is split at statement boundaries.

Prose. The system identifies paragraph boundaries (double newlines), section headings (Markdown headers, LaTeX section commands, HTML heading tags), and page breaks. Chunks are formed at section boundaries when possible, with paragraph boundaries as fallback.

Structured data. JSON arrays are split at element boundaries. CSV files are split at row boundaries, with the header row replicated in each chunk. YAML and TOML files are split at top-level key boundaries.

Configurable overlap. Adjacent chunks include a configurable overlap region (default: 64 tokens) to preserve context that spans boundaries. The overlap is included in both the preceding and following chunk, ensuring that queries about cross-boundary content match at least one chunk.

4.7 7. Filesystem Event Integration

The system optionally integrates with the operating system's filesystem event API:

macOS: FSEvents API, which provides a coalesced stream of filesystem changes at the directory level. The system registers a persistent event stream for the monitored directory tree. When events

arrive, the system computes content hashes only for files in the affected directories, rather than walking the entire tree.

Linux: inotify API, which provides per-file event notifications (IN_CREATE, IN_MODIFY, IN_DELETE, IN_MOVED_FROM, IN_MOVED_TO). The system adds recursive watches on the monitored directory tree. Inotify watch limits are configurable via `/proc/sys/fs/inotify/max_user_watches`.

Windows: ReadDirectoryChangesW API, which provides directory-level change notifications with file-level detail.

In all cases, the system falls back to full delta scanning when:

- The event queue overflows (events were generated faster than consumed).
- The system starts up and must reconcile state (events may have been missed while the system was not running).
- The user explicitly requests a full scan.

4.8 8. Zero Outbound Network Operation

The system is designed to operate with zero outbound network connections during normal indexing and search:

- Embedding models run locally using quantized model weights stored on disk.
- Vision models and LLMs run locally using the same infrastructure.
- The hash manifest, embedding cache, and encrypted index are stored on local disk.
- No telemetry, analytics, or crash reporting is transmitted.
- The system does not check for updates, download model weights, or contact any server during normal operation.

Network access may be required during initial setup (downloading model weights) but is neither required nor attempted during indexing, search, or any other normal operation.

4.9 9. Dual-Read Migration Mode

When transitioning from an unencrypted index to an encrypted one, the system enters a dual-read migration mode:

1. On read, the system first attempts to decrypt the record using the appropriate subkey.
2. If decryption succeeds, the record is already encrypted; proceed normally.
3. If decryption fails (the authentication tag does not verify), the system interprets the record as plaintext.
4. The plaintext record is encrypted under the appropriate subkey and written back, replacing the plaintext version.
5. The read operation returns the record contents to the caller.

This process is idempotent: re-reading an already-migrated record succeeds at step 2. Migration proceeds incrementally as records are accessed. A background sweep can optionally migrate all remaining plaintext records. After migration is complete (all records encrypted), the dual-read mode is disabled and all reads assume encrypted records, eliminating the decryption-failure branch.

The dual-read approach ensures:

- No downtime during migration.
- No separate migration tool or process.
- Interrupted migrations can be resumed without data loss.
- The system is functional throughout the migration, serving both encrypted and not-yet-encrypted records.

4.10 10. Search Operation

A semantic search query is processed as follows:

1. The query text is embedded using the same local embedding model used for indexing.
2. The vector index (stored encrypted under K_{vector}) is searched for nearest neighbors using cosine similarity or inner product distance.
3. Matching records' metadata (stored encrypted under K_{metadata}) is decrypted to retrieve file paths and contextual information.
4. Optionally, matching records' content (stored encrypted under K_{content}) is decrypted to provide text snippets.
5. Results are returned ranked by similarity score.

The search operation requires the master key (derived from the user's passphrase) to decrypt any results. Without the passphrase, the encrypted index reveals no information about the indexed content, file names, or file structure.

5 FIGURES

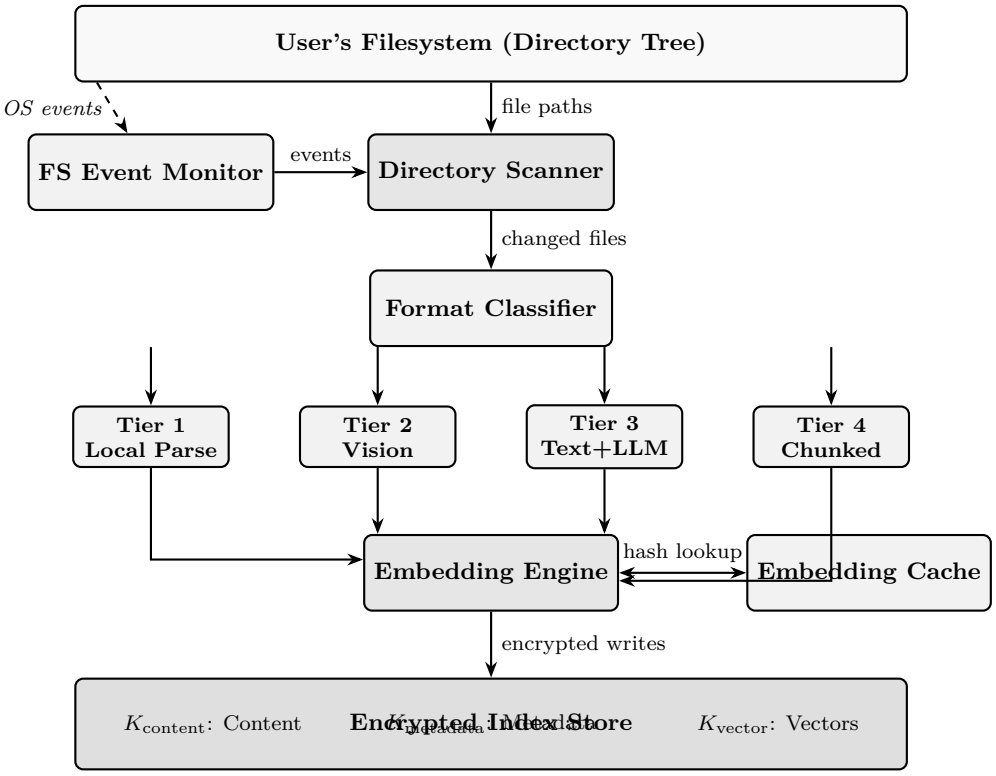

Figure 1: **System Architecture.** Six components: filesystem event monitor, directory scanner, format classifier, tiered processing pipeline, embedding engine with cache, and encrypted index store with three separately-keyed partitions. Changed files flow top-to-bottom; unchanged files are skipped entirely.

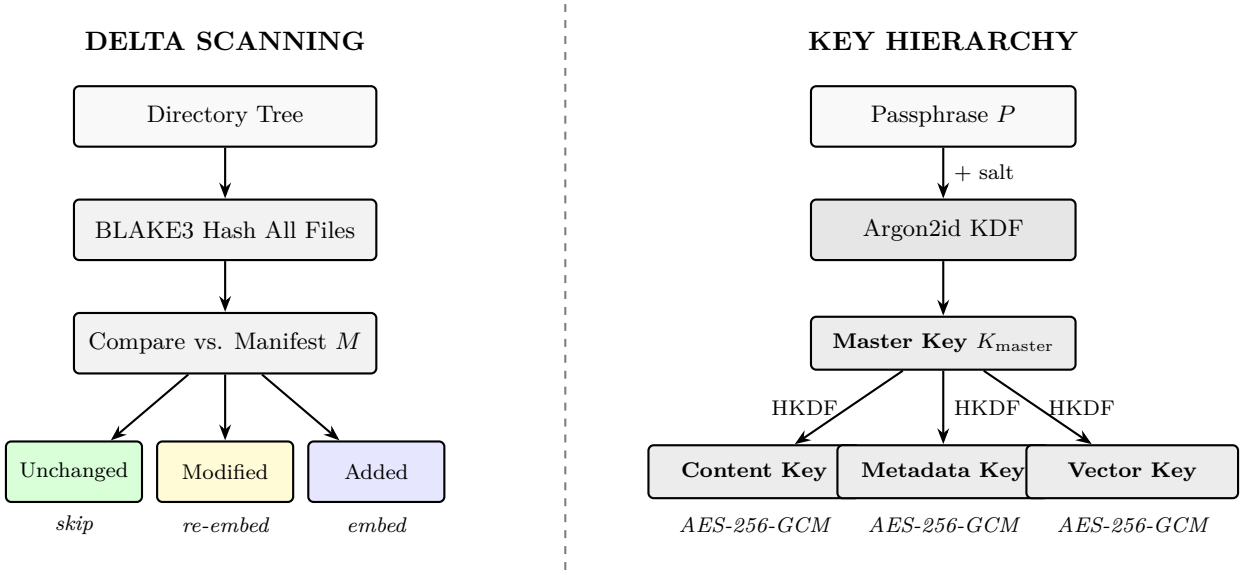

Figure 2: **Delta Scanning and Encryption Key Hierarchy.** Left: content-hash delta scanning classifies files as unchanged (skipped), modified (re-embedded), or added (embedded). Right: hierarchical key derivation from passphrase through Argon2id to master key, then HKDF to three purpose-specific subkeys, each using AES-256-GCM.

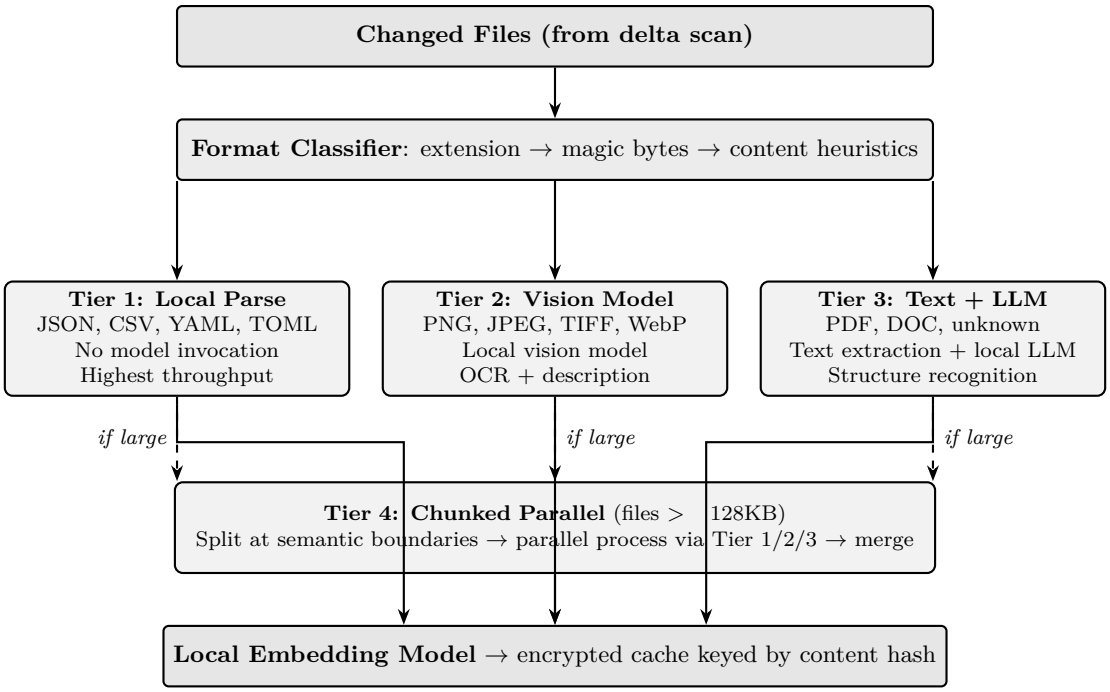

Figure 3: **Format-Aware Tiered Processing Pipeline.** Files are classified by format and routed to the appropriate tier. Tier 1 uses local parsers for structured data. Tier 2 uses a vision model for images. Tier 3 uses text extraction and LLM for PDFs. Tier 4 handles large files by chunking at semantic boundaries and processing in parallel. All tiers feed into a local embedding model.